

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Домашнее задание №4

Технический дизайн микросервисной архитектуры

по дисциплине «Программирование серверной логики»

Вариант: Сайт для поиска работы (Job Search Platform)

Студент: Гельм Даниил

Группа: К3340 / БР1.1

Санкт-Петербург

2026

Содержание

1. Исходный монолит
2. Разделение на микросервисы (database-per-service)
3. Архитектура: взаимосвязи и способы взаимодействия
4. Разделение базы данных
5. Межсервисные эндпоинты (OpenAPI), запросы, ответы и ошибки
6. План разделения монолита (шаги для ЛР2)
7. Выводы

1. Исходный монолит

Исходная реализация — лабораторная работа №1 (labs/lab1): одно Express-приложение, TypeORM, одна PostgreSQL. Все домены (Auth, Users, Companies, Resumes, Vacancies, Applications, Favorites, Skills) обслуживаются одним процессом и одной БД.

Публичное API описано в OpenAPI (Д32). Основной сценарий использования совпадает с Postman-сценарием Д33: регистрация работодателя и соискателя, компания, вакансия, резюме, поиск, отклик, избранное, смена статуса отклика.

2. Разделение на микросервисы (database-per-service)

Монолит разделён на независимые сервисы по доменным границам. Каждый сервис владеет своей базой данных (database-per-service). API Gateway сохраняет единую точку входа /api/v1 для клиента — публичный контракт не меняется.

Сервис	Порт	Собственная БД	Таблицы	Публичные маршруты (через Gateway)
API Gateway	3000	—	—	Все /api/v1/*
Auth Service	3001	auth_db	users	POST /auth/register, POST /auth/login, GET /users/me
Company Service	3002	company_db	companies	GET/POST /companies, GET/PATCH /companies/{id}
Skills Service	3003	skills_db	skills	GET/POST /skills
Resume Service	3004	resume_db	resumes, resume_skills	GET/POST /resumes, ...
Vacancy Service	3005	vacancy_db	vacancies, vacancy_skills	GET/POST /vacancies, ...
Application Service	3006	application_db	applications	POST /vacancies/{id}/applications, GET /applications/my,

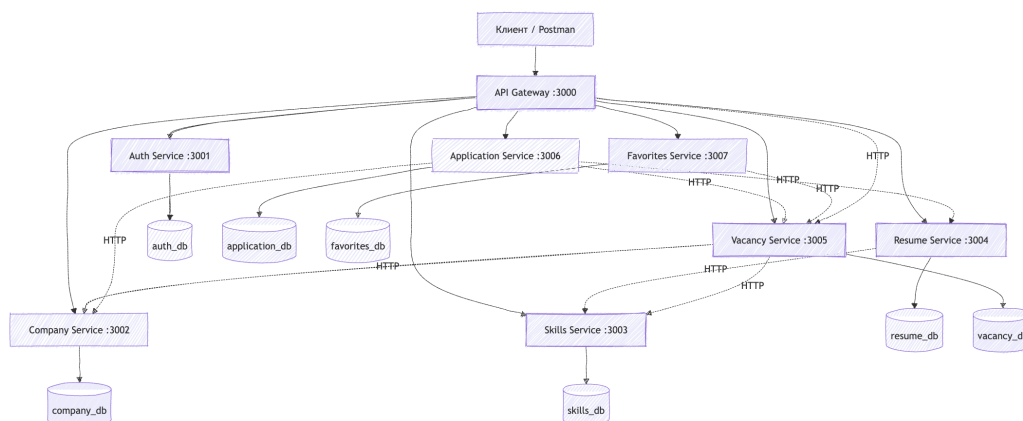
				PATCH .../status
Favorites Service	3007	favorites_db	favorite_vacancies	GET/POST/DELETE /vacancies/{id}/favorite

Skills Service выделен отдельно: справочник навыков используется и в резюме, и в вакансиях; при database-per-service таблица skills должна иметь одного владельца.

3. Архитектура: взаимосвязи и способы взаимодействия

3.1. Диаграмма взаимосвязи микросервисов

На рисунке 1 отражены все микросервисы, их базы данных и направления межсервисного взаимодействия (пунктир — HTTP между сервисами).



3.2. Способы взаимодействия

Клиент обращается только к API Gateway. Gateway проверяет JWT и проксирует запрос в нужный сервис. Между сервисами используется синхронное взаимодействие по HTTP/REST (internal API). Аутентификация service-to-service — заголовок X-Service-Token.

Кто вызывает	Кого	Когда	Протокол
Клиент	API Gateway	Любой публичный запрос	HTTP + JWT

API Gateway	Микросервис	Маршрутизация /api/v1	HTTP + JWT
Vacancy Service	Company Service	Создание вакансии: проверка companyId	HTTP internal
Application Service	Vacancy Service	Создание отклика: проверка вакансии	HTTP internal
Application Service	Resume Service	Создание отклика: проверка резюме и владельца	HTTP internal
Application Service	Company Service	Смена статуса: проверка прав работодателя	HTTP internal
Resume / Vacancy Service	Skills Service	Привязка skill id	HTTP internal
API Gateway	Favorites + Vacancy	GET /favorites: IDs → batch вакансий	HTTP internal

3.3. Маршрутизация API Gateway

Префикс публичного API	Целевой сервис
/api/v1/auth/*, /api/v1/users/*	Auth Service
/api/v1/companies/*	Company Service
/api/v1/skills/*	Skills Service
/api/v1/resumes/*	Resume Service
/api/v1/vacancies/* (кроме /applications, /favorite)	Vacancy Service
/api/v1/applications/*	Application Service
POST .../applications, .../favorite	Application / Favorites Service

4. Разделение базы данных

Каждая таблица монолита переносится в БД того сервиса, который владеет соответствующим доменом. Связи между сервисами — логические ссылки по ID (без FOREIGN KEY между разными базами).

БД	Таблицы	Логические ссылки на другие БД
auth_db	users	—
company_db	companies	owner_id → users.id
skills_db	skills	—
resume_db	resumes, resume_skills	user_id → users.id; skill_id → skills.id
vacancy_db	vacancies, vacancy_skills	company_id → companies.id; skill_id → skills.id
application_db	applications	vacancy_id → vacancies.id; resume_id → resumes.id
favorites_db	favorite_vacancies	user_id → users.id; vacancy_id → vacancies.id

Перед записью данных сервис проверяет существование внешних ID через internal HTTP (например, Vacancy Service вызывает Company Service при создании вакансии). Распределённые транзакции (2PC) не применяются.

4.1. Схемы таблиц по сервисам

БД	Ключевые поля таблиц
auth_db.users	id, email, password_hash, first_name, last_name, role
company_db.companies	id, owner_id, name, description, website, location
skills_db.skills	id, name
resume_db.resumes	id, user_id, title, about, experience_years, desired_salary

resume_db.resume_skills	resume_id, skill_id
vacancy_db.vacancies	id, company_id, title, salary_from, salary_to, location, ...
vacancy_db.vacancy_skills	vacancy_id, skill_id
application_db.applications	id, vacancy_id, resume_id, status, cover_letter
favorites_db.favorite_vacancies	user_id, vacancy_id, created_at

5. Межсервисные эндпоинты (OpenAPI), запросы, ответы и ошибки

Спецификация internal API оформлена в формате OpenAPI 3.0 и приложена к репозиторию: `homeworks/hw4/openapi/internal-apis.yaml` (см. Приложение Б). Эндпоинты недоступны клиенту — только для service-to-service и Gateway.

5.1. Перечень internal-эндпоинтов

Сервис	Метод и путь	Назначение
Auth	GET /internal/users/{userId}	Данные пользователя без password_hash
Auth	GET /internal/users/{userId}/exists	Проверка существования userId
Company	GET /internal/companies/{companyId}	Данные компании
Company	GET /internal/companies/{companyId}/owner	Получить ownerId
Skills	GET /internal/skills/{skillId}	Проверка навыка
Skills	POST /internal/skills/batch	Пакетное получение по списку id
Resume	GET /internal/resumes/{resumeId}	Данные резюме
Resume	GET /internal/resumes/by-user/{userId}	Резюме пользователя
Vacancy	GET /internal/vacancies/{vacancyId}	Данные вакансии

Vacancy	POST /internal/vacancies/batch	Пакет вакансий по id
Vacancy	GET /internal/vacancies/{id}/company-id	companyId вакансии
Application	POST /internal/applications	Создание отклика
Application	PATCH /internal/applications/{id}/status	Смена статуса
Favorites	GET /internal/favorites?userId=	Список vacancyId

5.2. Примеры запросов и ответов

Пример 1 — Application Service проверяет вакансию перед созданием отклика:

```
GET /internal/vacancies/5
X-Service-Token: <secret>

200 OK
{
  "id": 5,
  "companyId": 2,
  "title": "Backend Developer",
  ...
}
```

Пример 2 — Gateway создаёт отклик через Application Service:

```
POST /internal/applications
Content-Type: application/json

{
  "vacancyId": 5,
  "resumeId": 12,
  "userId": 7,
  "role": "applicant",
  "coverLetter": "..."
}

201 Created
{
  "id": 1,
  "vacancyId": 5,
  "resumeId": 12,
```

```
"status": "sent"
}
```

Пример 3 — Favorites Service возвращает id для агрегации на Gateway:

```
GET /internal/favorites?userId=7
```

```
200 OK
{
  "vacancyIds": [1, 5, 9]
}
```

5.3. Документирование ошибок

Формат тела ошибки (единый для публичного и internal API):

```
{
  "code": "NOT_FOUND",
  "message": "Вакансия не найдена"
}
```

HTTP	code	Описание
401	UNAUTHORIZED	Отсутствует или неверный JWT / X-Service-Token
403	FORBIDDEN	Нет прав (не владелец резюме / компании)
404	NOT_FOUND	Сущность не найдена в БД сервиса
409	CONFLICT	Дубликат (отклик, избранное, email)
422	VALIDATION_ERROR	Некорректное тело запроса
424	FAILED_DEPENDENCY	Зависимый сервис вернул ошибку
502	BAD_GATEWAY	Зависимый сервис недоступен / таймаут
500	INTERNAL_ERROR	Необработанная ошибка сервера

Полные схемы request/response и responses для каждого internal-эндпоинта описаны в openapi/internal-api.yaml (components/schemas, components/responses).

6. План разделения монолита (шаги для ЛР2)

Ниже — конкретные шаги реализации разделения монолита labs/lab1 на микросервисы в соответствии с настоящим документом.

Шаг	Действие
1	Создать структуру labs/lab2: gateway, services/*, packages/shared; настроить 7 баз PostgreSQL (или 7 database в одном инстансе для dev).
2	Реализовать Auth Service: перенести users, register, login, GET /users/me; добавить GET /internal/users/{id}.
3	Реализовать API Gateway: JWT, проксирование /auth и /users/me на Auth Service.
4	Реализовать Company Service и Skills Service; подключить маршруты через Gateway.
5	Реализовать Resume Service и Vacancy Service; при записи проверять skill_id и company_id через internal API.
6	Реализовать Application Service: создание отклика и смена статуса с цепочкой вызовов Vacancy, Resume, Company.
7	Реализовать Favorites Service; на Gateway — агрегация GET /favorites (favorites → batch vacancies).
8	Прогнать Postman-сценарий ДЗ3 через Gateway; убедиться, что публичный API работает как в

	МОНОЛИТЕ.
--	-----------

Структура каталогов для ЛР2: labs/lab2/gateway/, labs/lab2/services/{auth,company,skills,resume,vacancy,application,favorites}/, labs/lab2/packages/shared/.

7. Выводы

Выполнены требования ДЗ4: монолит Job Search Platform разделён на 7 микросервисов с принципом database-per-service; описаны взаимосвязи и HTTP-взаимодействие; спроектировано разделение БД; межсервисные эндпоинты задокументированы в OpenAPI; описаны форматы запросов, ответов и ошибок; сформирован пошаговый план реализации для ЛР2.

Публичный контракт /api/v1 сохраняется через API Gateway, что обеспечивает совместимость с результатами ДЗ2 и ДЗ3.